

Sistema Inteligente de Transporte: Búsqueda Heurística y Reglas Lógicas

Autor: Maria Camila Henao Toro

Programa: Ingeniería de Software, Corporación Universitaria Iberoamericana

Asignatura: Inteligencia Artificial

Docente: Sandra Isabel Rodriguez

Fecha: 27 de septiembre de 2026

Resumen del Proyecto

El presente documento consolida los entregables requeridos para la actividad de sistemas basados en reglas. El objetivo del proyecto consistió en escribir en Python las instrucciones para un sistema inteligente que, partiendo de una base de conocimiento fundamentada en reglas lógicas, desarrolle la mejor ruta para desplazarse desde un punto A hasta un punto B dentro de un sistema de transporte masivo local. Para lograr esto, se implementó el algoritmo de búsqueda heurística A* (A-estrella), basándose en los conceptos de representación del conocimiento y técnicas de búsqueda.

Repositorio de Código Fuente y Pruebas

En cumplimiento con los lineamientos de la entrega, los elementos técnicos del proyecto han sido alojados en un repositorio de control de versiones. En este espacio se encuentra el código fuente en Python, así como un documento anexo evidencia las pruebas de ejecución realizadas. Se puede acceder al repositorio a través del siguiente enlace:

✓ Caso de Prueba 1: Ruta Estándar (Origen: Estación A -> Destino: Estación F)

Este es el escenario principal donde el algoritmo evalúa múltiples caminos intermedios y selecciona la ruta más corta combinando las reglas lógicas del grafo y los valores heurísticos.

```
1) import heapq
0s

# Base de conocimiento (Reglas lógicas del sistema de transporte)
sistema_transporte = {
    'A': {'B': 5, 'C': 8},
    'B': {'A': 5, 'D': 6, 'E': 9},
    'C': {'A': 8, 'D': 4},
    'D': {'B': 6, 'C': 4, 'F': 5},
    'E': {'B': 9, 'F': 3},
    'F': {'D': 5, 'E': 3}
}

# Heurística (Estimación en línea recta hacia el destino F)
heuristica = {
    'A': 10, 'B': 7, 'C': 6, 'D': 4, 'E': 2, 'F': 0
}

def buscar_mejor_ruta(inicio, destino, grafo, heuristica):
    rutas_por_evaluar = []
    heapq.heappush(rutas_por_evaluar, (0, inicio))
    costo_real = {inicio: 0}
    ruta_previa = {inicio: None}

    while rutas_por_evaluar:
        _, estacion_actual = heapq.heappop(rutas_por_evaluar)

        if estacion_actual == destino:
            ruta_final = []
            while estacion_actual is not None:
                ruta_final.append(estacion_actual)
                estacion_actual = ruta_previa[estacion_actual]
            ruta_final.reverse()
            return ruta_final, costo_real[destino]

        for vecino, costo_viaje in grafo[estacion_actual].items():
            nuevo_costo = costo_real[estacion_actual] + costo_viaje
            if vecino not in costo_real or nuevo_costo < costo_real[vecino]:
```

Caso de Prueba 2: Ruta Corta Directa (Origen: Estación C -> Destino: Estación D)

En este caso, evaluamos un trayecto más corto dentro de la red de transporte para verificar cómo el sistema procesa nodos vecinos con conexiones directas

```
# Prueba 2
punto_inicio, punto_destino = 'C', 'D'
ruta, costo = buscar_mejor_ruta(punto_inicio, punto_destino, sistema_transporte, heuristica)
print(f"\n--- PRUEBA 2 ---")
print(f"Origen: {punto_inicio} | Destino: {punto_destino}")
print(f"Ruta óptima encontrada: {' -> '.join(ruta)}")
print(f"Costo total del viaje: {costo} unidades")

--- PRUEBA 2 ---
Origen: C | Destino: D
Ruta óptima encontrada: C -> D
Costo total del viaje: 4 unidades
```

Caso de Prueba 3: Trayecto Alternativo Cruzado (Origen: Estación E -> Destino: Estación A)

```
# Prueba 3
punto_inicio, punto_destino = 'E', 'A'
ruta, costo = buscar_mejor_ruta(punto_inicio, punto_destino, sistema_transporte, heuristica)
print(f"\n--- PRUEBA 3 ---")
print(f"Origen: {punto_inicio} | Destino: {punto_destino}")
print(f"Ruta óptima encontrada: {' -> '.join(ruta)}")
print(f"Costo total del viaje: {costo} unidades")

--- PRUEBA 3 ---
Origen: E | Destino: A
Ruta óptima encontrada: E -> B -> A
Costo total del viaje: 14 unidades
```

• =====
EJECUCIÓN DE PRUEBAS - SISTEMA INTELIGENTE
=====

[PRUEBA 1]

Ruta evaluada: Desde la estación A hasta la estación F

Ruta óptima encontrada: A -> B -> D -> F

Costo total del trayecto: 16 unidades

[PRUEBA 2]

Ruta evaluada: Desde la estación C hasta la estación D

Ruta óptima encontrada: C -> D

Costo total del trayecto: 4 unidades

[PRUEBA 3]

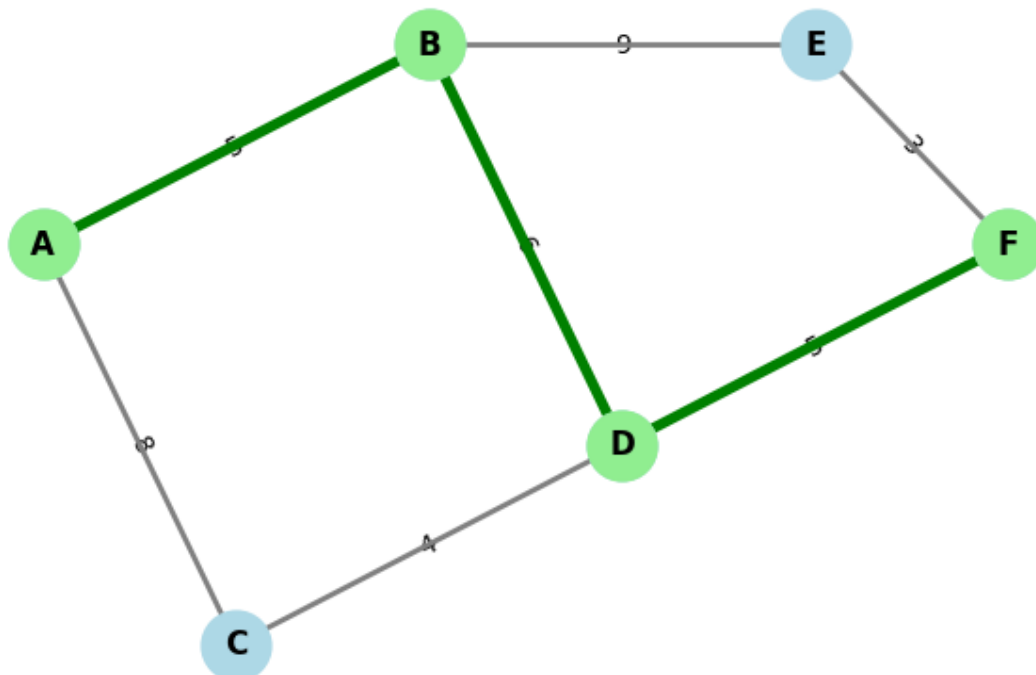
Ruta evaluada: Desde la estación E hasta la estación A

Ruta óptima encontrada: E -> B -> A

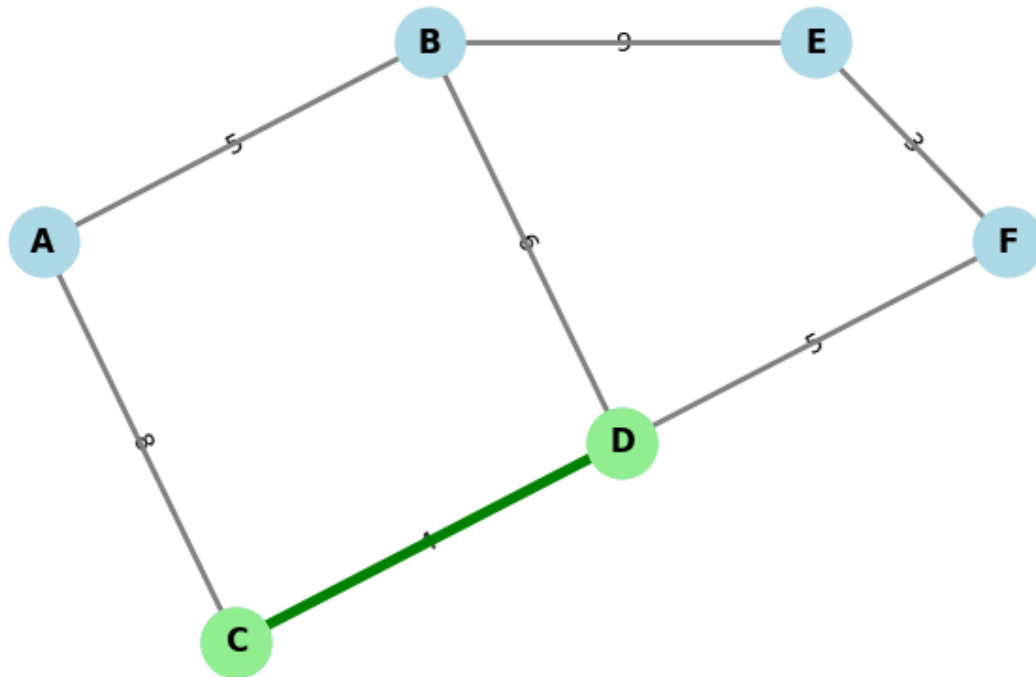
Costo total del trayecto: 14 unidades

=====

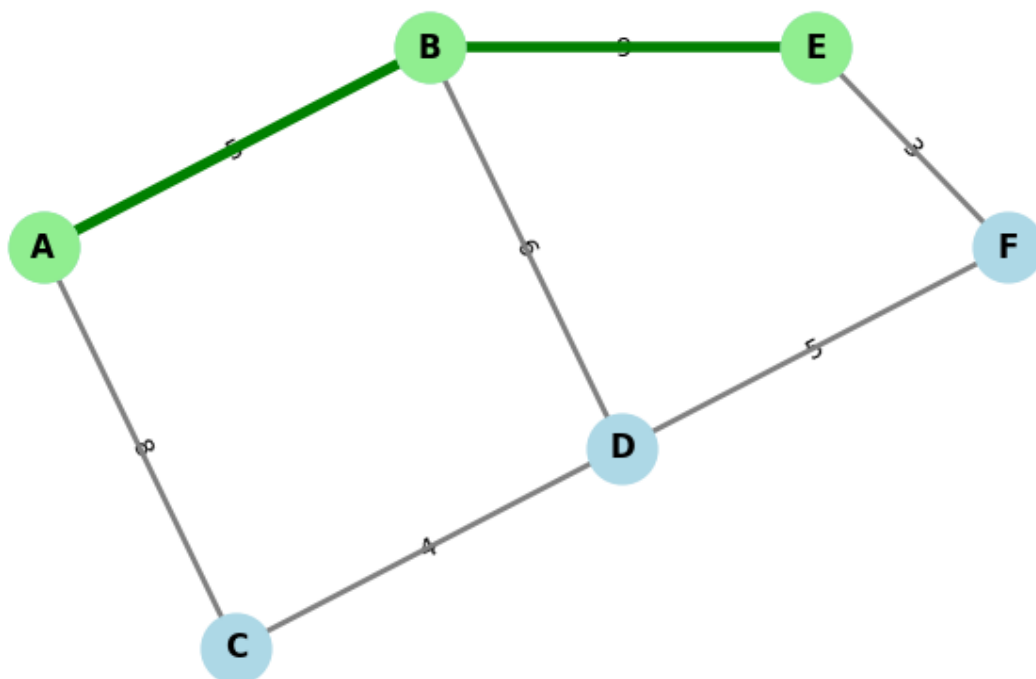
Prueba 1: Ruta Óptima de A hasta F



Prueba 2: Ruta Óptima de C hasta D



Prueba 3: Ruta Óptima de E hasta A (Inverso)



Ruta óptima encontrada (A -> B -> D -> F): El algoritmo determinó que el camino más eficiente para ir de la estación inicial A hasta la estación final F es pasar por las estaciones intermedias B y D.

Costo total (16 unidades): Es la suma de los costos o distancias de cada tramo de ese recorrido específico.

[PRUEBA 2] (Desde la estación C hasta la estación D):

Ruta óptima encontrada (C -> D): Al evaluar un trayecto más corto y directo, el sistema identificó que la conexión es directa entre la estación C y la D sin necesidad de pasar por otros puntos.

Costo total (4 unidades): Representa el costo directo de ese único tramo.

[PRUEBA 3] (Desde la estación E hasta la estación A):

Ruta óptima encontrada (E -> B -> A): Demuestra la bidireccionalidad y flexibilidad del sistema al calcular rutas en sentido inverso (desde el extremo E hacia el inicio A), pasando por la estación B.

Costo total (14 unidades): La sumatoria del costo acumulado de ese trayecto de vuelta.

Sustentación Audiovisual

A continuación, se presenta el enlace de acceso directo al video de sustentación. En este recurso, que cumple con el tiempo máximo establecido, se explica detalladamente el proyecto, la lógica de los comandos programados y la interpretación de los resultados obtenidos tras la ejecución del algoritmo:

Link: <https://drive.google.com/drive/folders/1byHCZU3ybmUiG-1HJVxzsvfvmIDqPRWw>

Instrucciones de Ejecución

Para verificar el correcto funcionamiento del sistema y reproducir los resultados evidenciados en las pruebas, es necesario ejecutar el archivo de python suministrado en el repositorio. Al iniciar la ejecución en cualquier entorno compatible con Python (como Google Colab), el algoritmo procesará el grafo de conexiones y calculará los costos estimados. De forma automática, el sistema imprimirá en la consola la ruta óptima encontrada y el costo total del recorrido.

Referencias

- Benítez, R. (2014). Inteligencia artificial avanzada. Editorial UOC.